

1. File Handling - Read and Write a Text File

Problem Statement:

Write a Java program that reads the contents of a text file and writes it into a new file. If the source file does not exist, display an appropriate message.

Requirements:

- Use `FileInputStream` and `FileOutputStream`.
- Handle `IOException` properly.
- Ensure that the destination file is created if it does not exist.

```
package com.filehandling.readwrite;

import java.io.*;

public class FileHandler {

    // Method to read content from a source file and write it to a destination file
    public void copyFile(String sourceFile, String destinationFile) {
        FileInputStream fis = null;
        FileOutputStream fos = null;

        try {
            // Creating a File object for the source file
            File source = new File(sourceFile);

            // Checking if the source file exists
            if (!source.exists()) {
                System.out.println("Source file does not exist.");
                return;
            }

            // Initializing FileInputStream to read from the source file
            fis = new FileInputStream(source);

            // Initializing FileOutputStream to write to the destination file
            fos = new FileOutputStream(destinationFile);
```

```

        int bytesRead;

        // Reading data from the source file and writing it to the destination
file
        while ((bytesRead = fis.read()) != -1) {
            fos.write(bytesRead);
        }

        // Displaying success message after writing to the destination file
        System.out.println("File copied successfully to " + destinationFile);
    } catch (IOException e) {
        // Handling any IOException that may occur
        System.out.println("An error occurred: " + e.getMessage());
    } finally {
        try {
            // Closing FileInputStream if it is not null
            if (fis != null) {
                fis.close();
            }

            // Closing FileOutputStream if it is not null
            if (fos != null) {
                fos.close();
            }
        } catch (IOException e) {
            System.out.println("Failed to close streams: " + e.getMessage());
        }
    }
}
}
}

```

```

package com.filehandling.readwrite;

public class Main {
    public static void main(String[] args) {
        // Creating an instance of FileHandler
        FileHandler fileHandler = new FileHandler();

        // Specifying the source and destination file paths
        String sourceFile =
"src\\main\\java\\com\\filehandling\\readwrite\\source.txt";
    }
}

```

```
String destinationFile =  
"src\\main\\java\\com\\filehandling\\readwrite\\destination.txt";  
  
// Calling the copyFile method to copy the file  
fileHandler.copyFile(sourceFile, destinationFile);  
}  
}
```

2. Buffered Streams - Efficient File Copy

Problem Statement:

Create a Java program that copies a large file (e.g., 100MB) from one location to another using **Buffered Streams** (`BufferedInputStream` and `BufferedOutputStream`). Compare the performance with normal file streams.

Requirements:

- Read and write in chunks of **4 KB (4096 bytes)**.
- Use `System.nanoTime()` to measure execution time.
- Compare execution time with **unbuffered streams**.

```
package com.filehandling.buffercopy;  
  
import java.io.*;  
  
public class BufferedFileCopy {  
  
    // Method to copy a file using FileInputStream and FileOutputStream (unbuffered)  
    public void copyWithFileStreams(String sourceFile, String destinationFile) {  
        FileInputStream fis = null;
```

```

FileOutputStream fos = null;

try {
    // Creating FileInputStream and FileOutputStream
    fis = new FileInputStream(sourceFile);
    fos = new FileOutputStream(destinationFile);

    // Creating a buffer to read and write data
    byte[] buffer = new byte[4096]; // 4 KB buffer
    int bytesRead;

    // Measuring start time
    long startTime = System.nanoTime();

    // Reading and writing data in chunks
    while ((bytesRead = fis.read(buffer)) != -1) {
        fos.write(buffer, 0, bytesRead);
    }

    // Measuring end time and displaying execution time
    long endTime = System.nanoTime();
    System.out.println("Unbuffered copy completed in " + (endTime -
startTime) / 1_000_000 + " ms");
    } catch (IOException e) {
        System.out.println("An error occurred: " + e.getMessage());
    } finally {
        try {
            if (fis != null) fis.close();
            if (fos != null) fos.close();
        } catch (IOException e) {
            System.out.println("Failed to close streams: " + e.getMessage());
        }
    }
}

// Method to copy a file using BufferedInputStream and BufferedOutputStream
(buffered)
public void copyWithBufferedStreams(String sourceFile, String destinationFile) {
    BufferedInputStream bis = null;
    BufferedOutputStream bos = null;

    try {
        // Creating BufferedInputStream and BufferedOutputStream
        bis = new BufferedInputStream(new FileInputStream(sourceFile));
        bos = new BufferedOutputStream(new FileOutputStream(destinationFile));
    }

```

```
// Creating a buffer to read and write data
byte[] buffer = new byte[4096]; // 4 KB buffer
int bytesRead;

// Measuring start time
long startTime = System.nanoTime();

// Reading and writing data in chunks
while ((bytesRead = bis.read(buffer)) != -1) {
    bos.write(buffer, 0, bytesRead);
}

// Measuring end time and displaying execution time
long endTime = System.nanoTime();
System.out.println("Buffered copy completed in " + (endTime - startTime)
/ 1_000_000 + " ms");
} catch (IOException e) {
    System.out.println("An error occurred: " + e.getMessage());
} finally {
    try {
        if (bis != null) bis.close();
        if (bos != null) bos.close();
    } catch (IOException e) {
        System.out.println("Failed to close streams: " + e.getMessage());
    }
}
}
```

```
package com.filehandling.buffercopy;

import com.filehandling.buffercopy.BufferedFileCopy;

public class Main {
    public static void main(String[] args) {
        // Creating an instance of BufferedFileCopy
        BufferedFileCopy fileCopy = new BufferedFileCopy();

        // Specifying the source and destination file paths
        String sourceFile =
"src\\main\\java\\com\\filehandling\\buffercopy\\largefile.txt";
    }
}
```

```
String destinationFile1 =  
"src\\main\\java\\com\\filehandling\\buffercopy\\destination_unbuffered.txt";  
String destinationFile2 =  
"src\\main\\java\\com\\filehandling\\buffercopy\\destination_buffered.txt";  
  
// Copying file using unbuffered streams and measuring time  
System.out.println("Copying using unbuffered streams...");  
fileCopy.copyWithFileStreams(sourceFile, destinationFile1);  
  
// Copying file using buffered streams and measuring time  
System.out.println("Copying using buffered streams...");  
fileCopy.copyWithBufferedStreams(sourceFile, destinationFile2);  
}  
}
```

3. Read User Input from Console

Problem Statement:

Write a program that asks the user for their **name, age, and favorite programming language**, then saves this information into a file.

Requirements:

- Use **BufferedReader** for console input.
- Use **FileWriter** to write the data into a file.
- Handle exceptions properly.

```
package com.filehandling.consoleinput;  
  
import java.io.*;  
  
public class UserInputToFile {
```

```
public static void main(String[] args) {
    BufferedReader reader = null;
    FileWriter writer = null;

    try {
        // Creating a BufferedReader to read input from the console
        reader = new BufferedReader(new InputStreamReader(System.in));

        // Reading user's name
        System.out.print("Enter your name: ");
        String name = reader.readLine();

        // Reading user's age
        System.out.print("Enter your age: ");
        int age = Integer.parseInt(reader.readLine());

        // Reading user's favorite programming language
        System.out.print("Enter your favorite programming language: ");
        String language = reader.readLine();

        // Creating a FileWriter to write user details into a file
        writer = new
FileWriter("C:\\Users\\Lenovo\\Desktop\\capgemini\\classroom\\week04-copy\\day03_st
reams\\src\\main\\java\\com\\filehandling\\consoleinput\\UserDetails.txt");
        writer.write("User Details:\n");
        writer.write("Name: " + name + "\n");
        writer.write("Age: " + age + "\n");
        writer.write("Favorite Programming Language: " + language + "\n");

        // Confirmation message after writing to the file
        System.out.println("User details have been saved to UserDetails.txt");

    } catch (IOException e) {
        // Handling IOException
        System.out.println("An error occurred: " + e.getMessage());
    } catch (NumberFormatException e) {
        // Handling invalid age input
        System.out.println("Invalid input for age. Please enter a valid
integer.");
    } finally {
        // Closing resources
        try {
            if (reader != null) reader.close();
            if (writer != null) writer.close();
        }
    }
}
```

```
    } catch (IOException e) {  
        System.out.println("Failed to close resources: " + e.getMessage());  
    }  
}  
}
```

4. Serialization - Save and Retrieve an Object

Problem Statement:

Design a Java program that allows a user to **store a list of employees in a file** using **Object Serialization** and later retrieve the data from the file.

Requirements:

- Create an **Employee** class with fields: **id**, **name**, **department**, **salary**.
- Serialize the list of employees into a file (**ObjectOutputStream**).
- Deserialize and display the employees from the file (**ObjectInputStream**).
- Handle **ClassNotFoundException** and **IOException**.

```
package com.filehandling.serialization;  
  
import java.io.Serializable;  
  
// Employee class representing an employee's details  
// Implements Serializable to allow object serialization  
public class Employee implements Serializable {  
    private static final long serialVersionUID = 1L;  
}
```



```
// Employee fields: id, name, department, and salary
private int id;
private String name;
private String department;
private double salary;

// Constructor to initialize Employee fields
public Employee(int id, String name, String department, double salary) {
    this.id = id;
    this.name = name;
    this.department = department;
    this.salary = salary;
}

// Getter methods to access employee details
public int getId() {
    return id;
}

public String getName() {
    return name;
}

public String getDepartment() {
    return department;
}

public double getSalary() {
    return salary;
}

// Overriding the toString method for displaying employee details
@Override
public String toString() {
    return "Employee [ID=" + id + ", Name=" + name + ", Department=" +
department + ", Salary=" + salary + "]\n";
}
}
```

```
package com.filehandling.serialization;

import java.io.*;
import java.util.ArrayList;
import java.util.List;

public class Main {

    public static void main(String[] args) {
        // Creating a list of employees with sample data
        List<Employee> employeeList = new ArrayList<>();
        employeeList.add(new Employee(1, "Raj", "IT", 60000));
        employeeList.add(new Employee(2, "Gagan", "HR", 55000));
        employeeList.add(new Employee(3, "Shubham", "Finance", 50000));

        // File name for storing serialized data
        String fileName =
"C:\\Users\\Lenovo\\Desktop\\capgemini\\classroom\\week04-copy\\day03_streams\\src\\
\\main\\java\\com\\filehandling\\serialization\\employees.ser";

        // Serializing the employee list to a file
        serializeEmployeeList(employeeList, fileName);

        // Deserializing the employee list from the file
        List<Employee> deserializedList = deserializeEmployeeList(fileName);

        // Displaying the deserialized employee list
        System.out.println("Deserialized Employee List:");
        for (Employee employee : deserializedList) {
            System.out.println(employee);
        }
    }

    // Method to serialize the list of employees
    private static void serializeEmployeeList(List<Employee> employeeList, String
fileName) {
        try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream(fileName))) {
            // Writing the employee list to the file
            oos.writeObject(employeeList);
            System.out.println("Employee list has been serialized to " + fileName);
        } catch (IOException e) {
            // Handling IOException during serialization
            System.out.println("Error during serialization: " + e.getMessage());
        }
    }
}
```

```

    }

    // Method to deserialize the list of employees from the file
    private static List<Employee> deserializeEmployeeList(String fileName) {
        List<Employee> employeeList = null;
        try (ObjectInputStream ois = new ObjectInputStream(new
        FileInputStream(fileName))) {
            // Reading the employee list from the file
            employeeList = (List<Employee>) ois.readObject();
        } catch (IOException | ClassNotFoundException e) {
            // Handling IOException and ClassNotFoundException during
            deserialization
            System.out.println("Error during deserialization: " + e.getMessage());
        }
        return employeeList;
    }
}

```

5. ByteArray Stream - Convert Image to ByteArray

Problem Statement:

Write a Java program that **converts an image file into a byte array** and then writes it back to another image file.

Requirements:

- Use **ByteArrayInputStream** and **ByteArrayOutputStream**.
- Verify that the new file is identical to the original image.
- Handle **IOException**.

```

package com.filehandling.imagetobytearray;

import java.io.*;

public class ImageConverter {

    public static void main(String[] args) {

```

```
// Defining the source and destination file paths
String sourceImage =
"C:\\Users\\Lenovo\\Desktop\\capgemini\\classroom\\week04-copy\\day03_streams\\src\\
\\main\\java\\com\\filehandling\\imagetobytearray\\original_image.jpg";
String destinationImage =
"C:\\Users\\Lenovo\\Desktop\\capgemini\\classroom\\week04-copy\\day03_streams\\src\\
\\main\\java\\com\\filehandling\\imagetobytearray\\copied_image.jpg";

// Converting the image file to a byte array
byte[] imageData = convertImageToByteArray(sourceImage);

// Checking if the byte array is not null
if (imageData != null) {
    // Writing the byte array back to a new image file
    writeByteArrayToImage(imageData, destinationImage);
}
}

// Method for converting an image file to a byte array
private static byte[] convertImageToByteArray(String filePath) {

    // Creating a FileInputStream to read the image file
    // Creating a ByteArrayOutputStream to store the byte array
    try (FileInputStream fis = new FileInputStream(filePath);
        ByteArrayOutputStream baos = new ByteArrayOutputStream()) {

        // Reading the image file in chunks and writing to ByteArrayOutputStream
        byte[] buffer = new byte[4096]; // Buffer size of 4 KB
        int bytesRead;
        while ((bytesRead = fis.read(buffer)) != -1) {
            baos.write(buffer, 0, bytesRead);
        }

        // Returning the byte array
        System.out.println("Image successfully converted to byte array.");
        return baos.toByteArray();
    } catch (IOException e) {
        // Handling IOException
        System.out.println("Error while converting image to byte array: " +
e.getMessage());
        return null;
    }
}
```

```
// Method for writing a byte array back to an image file
private static void writeByteArrayToImage(byte[] imageData, String filePath) {

    // Creating a ByteArrayInputStream from the byte array
    // Creating a FileOutputStream to write the new image file
    try (ByteArrayInputStream bais = new ByteArrayInputStream(imageData);
        FileOutputStream fos = new FileOutputStream(filePath)) {

        // Reading from ByteArrayInputStream and writing to the new image file
        byte[] buffer = new byte[4096]; // Buffer size of 4 KB
        int bytesRead;
        while ((bytesRead = bais.read(buffer)) != -1) {
            fos.write(buffer, 0, bytesRead);
        }

        // Printing success message
        System.out.println("Byte array successfully written to " + filePath);
    } catch (IOException e) {
        // Handling IOException during writing
        System.out.println("Error while writing byte array to image: " +
            e.getMessage());
    }
}
```

6. Filter Streams - Convert Uppercase to Lowercase

Problem Statement:

Create a program that reads a text file and writes its contents into another file, converting all uppercase letters to lowercase.

Requirements:

- Use `FileReader` and `FileWriter`.
- Use `BufferedReader` and `BufferedWriter` for efficiency.

- Handle character encoding issues.

```
package com.filehandling.filestream;

import java.io.*;

public class UppercaseToLowercaseConverter {

    public static void main(String[] args) {

        // Defining the source and destination file paths
        String sourceFile =
"C:\\Users\\Lenovo\\Desktop\\capgemini\\classroom\\week04-copy\\day03_streams\\src\\
\\main\\java\\com\\filehandling\\filestream\\source.txt";
        String destinationFile =
"C:\\Users\\Lenovo\\Desktop\\capgemini\\classroom\\week04-copy\\day03_streams\\src\\
\\main\\java\\com\\filehandling\\filestream\\destination.txt";

        // Converting all uppercase letters in the source file to lowercase and
        writing to the destination file
        convertUppercaseToLowercase(sourceFile, destinationFile);
    }

    // Method for converting uppercase letters to lowercase and writing to a new
    file
    private static void convertUppercaseToLowercase(String sourceFile, String
destinationFile) {
        // Creating FileReader and BufferedReader to read from the source file
        try (FileReader fr = new FileReader(sourceFile);
            BufferedReader br = new BufferedReader(fr);
            // Creating FileWriter and BufferedWriter to write to the
            destination file

            FileWriter fw = new FileWriter(destinationFile);
            BufferedWriter bw = new BufferedWriter(fw)) {

            // Reading the source file line by line
            String line;
            while ((line = br.readLine()) != null) {
                // Converting the line to lowercase
                line = line.toLowerCase();

                // Writing the lowercase line to the destination file
                bw.write(line);
                bw.newLine();
            }
        }
    }
}
```

```
// Printing success message
System.out.println("Successfully converted uppercase to lowercase and
written to " + destinationFile);

} catch (IOException e) {
    // Handling IOException
    System.out.println("Error during file processing: " + e.getMessage());
}
}
```

7. Data Streams - Store and Retrieve Primitive Data

Problem Statement:

Write a Java program that stores **student details** (roll number, name, GPA) in a binary file and retrieves it later.

Requirements:

- Use `DataOutputStream` to write primitive data.
- Use `DataInputStream` to read data.
- Ensure proper closing of resources.

```
package com.filehandling.datastreams;

import java.io.*;
```

```
public class StudentDataHandler {

    public static void main(String[] args) {

        // Defining the file name to store student data
        String fileName =
"C:\\Users\\Lenovo\\Desktop\\capgemini\\classroom\\week04-copy\\day03_streams\\src\\
\\main\\java\\com\\filehandling\\datastreams\\student_data.bin";

        // Storing student details into the binary file
        storeStudentData(fileName);

        // Retrieving and displaying student details from the binary file
        retrieveStudentData(fileName);
    }

    // Method for storing student data into a binary file using DataOutputStream
    private static void storeStudentData(String fileName) {
        try (FileOutputStream fos = new FileOutputStream(fileName);
            DataOutputStream dos = new DataOutputStream(fos)) {

            // Writing student details (roll number, name, GPA) to the binary file
            dos.writeInt(101);           // Roll number
            dos.writeUTF("Raj Verma");   // Name
            dos.writeDouble(8.5);        // GPA

            dos.writeInt(102);           // Roll number
            dos.writeUTF("Shubham Jain"); // Name
            dos.writeDouble(9.1);        // GPA

            dos.writeInt(103);           // Roll number
            dos.writeUTF("Deepanshu Malviya"); // Name
            dos.writeDouble(8.8);        // GPA

            // Printing success message
            System.out.println("Student data successfully stored in " + fileName);

        } catch (IOException e) {
            // Handling IOException
            System.out.println("Error while storing student data: " +
e.getMessage());
        }
    }
}
```



```
// Method for retrieving student data from the binary file using DataInputStream
private static void retrieveStudentData(String fileName) {
    try (FileInputStream fis = new FileInputStream(fileName);
        DataInputStream dis = new DataInputStream(fis)) {

        // Reading and displaying student details until the end of the file
        System.out.println("Retrieving student data from " + fileName + ":");
        while (dis.available() > 0) {
            int rollNumber = dis.readInt();           // Reading roll number
            String name = dis.readUTF();              // Reading name
            double gpa = dis.readDouble();            // Reading GPA

            // Printing student details
            System.out.println("Roll No: " + rollNumber + ", Name: " + name + ",
GPA: " + gpa);
        }

        } catch (IOException e) {
            // Handling IOException
            System.out.println("Error while retrieving student data: " +
e.getMessage());
        }
    }
}
```

8. Piped Streams - Inter-Thread Communication

Problem Statement:

Implement a Java program where one thread **writes data** into a **PipedOutputStream** and another thread **reads data** from a **PipedInputStream**.

Requirements:

- Use **two threads** for reading and writing.

- Synchronize properly to prevent data loss.
- Handle `IOException`.

```
package com.filehandling.pipedstreams;

import java.io.*;

public class PipedStreamCommunication {
    public static void main(String[] args) {
        // Creating piped input and output streams for communication between threads
        try (PipedOutputStream pos = new PipedOutputStream();
            PipedInputStream pis = new PipedInputStream(pos)) {

            // Creating and starting writer thread
            Thread writerThread = new Thread(new WriterTask(pos), "WriterThread");
            writerThread.start();

            // Creating and starting reader thread
            Thread readerThread = new Thread(new ReaderTask(pis), "ReaderThread");
            readerThread.start();

            // Joining threads to ensure the main thread waits for their completion
            writerThread.join();
            readerThread.join();

        } catch (IOException | InterruptedException e) {
            // Handling exceptions
            System.out.println("Error: " + e.getMessage());
        }
    }
}

// Writer task for writing data into the PipedOutputStream
class WriterTask implements Runnable {
    private PipedOutputStream pos;

    // Initializing the PipedOutputStream
    public WriterTask(PipedOutputStream pos) {
        this.pos = pos;
    }

    @Override
    public void run() {
        try {
```

```
// Writing data to the PipedOutputStream
for (int i = 1; i <= 5; i++) {
    String message = "Message " + i;
    pos.write(message.getBytes());
    System.out.println("Writer: Sent - " + message);
    Thread.sleep(500); // Simulating delay
}
} catch (IOException e) {
    // Handling exceptions
    System.out.println("Writer Error: " + e.getMessage());
} catch ( InterruptedException e) {
    // Handling exceptions
    System.out.println("Writer Error: " + e.getMessage());
} finally {
    try {
        pos.close(); // Closing the stream
    } catch (IOException e) {
        System.out.println("Error closing writer: " + e.getMessage());
    }
}
}

// Reader task for reading data from the PipedInputStream
class ReaderTask implements Runnable {
    private PipedInputStream pis;

    // Initializing the PipedInputStream
    public ReaderTask(PipedInputStream pis) {
        this.pis = pis;
    }

    @Override
    public void run() {
        try {
            // Reading data from the PipedInputStream
            byte[] buffer = new byte[1024];
            int bytesRead;
            while ((bytesRead = pis.read(buffer)) != -1) {
                String message = new String(buffer, 0, bytesRead);
                System.out.println("Reader: Received - " + message);
            }
        } catch (IOException e) {
            // Handling exceptions
            System.out.println("Reader Error: " + e.getMessage());
        }
    }
}
```

```

    } finally {
        try {
            pis.close(); // Closing the stream
        } catch (IOException e) {
            System.out.println("Error closing reader: " + e.getMessage());
        }
    }
}
}

```

9. Read a Large File Line by Line

Problem Statement:

Develop a Java program that efficiently reads a **large text file** (500MB+) **line by line** and prints only lines containing the word **"error"**.

Requirements:

- Use **BufferedReader** for efficient reading.
- Read line-by-line instead of loading the entire file.
- Display only lines containing **"error"** (case insensitive).

```

package com.filehandling.largefilereader;

import java.io.*;

public class LargeFileReader {
    public static void main(String[] args) {
        // Specifying the path of the large text file
        String filePath =

```

```
"C:\\Users\\Lenovo\\Desktop\\capgemini\\classroom\\week04-copy\\day03_streams\\src\\
\\main\\java\\com\\filehandling\\largefilereader\\large.txt";

// Reading the file line by line
try (BufferedReader reader = new BufferedReader(new FileReader(filePath))) {
    String line;
    while ((line = reader.readLine()) != null) {
        // Checking if the line contains the word "error" (case insensitive)
        if (line.toLowerCase().contains("error")) {
            // Printing the line containing "error"
            System.out.println(line);
        }
    }
} catch (IOException e) {
    // Handling exceptions related to file operations
    System.out.println("Error reading the file: " + e.getMessage());
}
}
```

10. Count Words in a File

Problem Statement:

Write a Java program that **counts the number of words in a given text file** and displays the **top 5 most frequently occurring words**.

Requirements:

- Use `FileReader` and `BufferedReader` to read the file.
- Use a `HashMap<String, Integer>` to count word occurrences.
- Sort the words based on frequency and display the top 5.

```
package com.filehandling.wordcounter;

import java.io.*;
import java.util.*;

public class WordFrequencyCounter {
    public static void main(String[] args) {
        // Specifying the file path
        String filePath =
"C:\\Users\\Lenovo\\Desktop\\capgemini\\classroom\\week04-copy\\day03_streams\\src\\
\\main\\java\\com\\filehandling\\wordcounter\\large.txt";

        // Creating a HashMap to store word frequencies
        Map<String, Integer> wordCountMap = new HashMap<>();

        // Reading the file and counting word occurrences
        try (BufferedReader reader = new BufferedReader(new FileReader(filePath))) {
            String line;
            while ((line = reader.readLine()) != null) {
                // Splitting the line into words using spaces and punctuation as
                // delimiters
                String[] words = line.toLowerCase().split("[\\s.,!?:;()\"'-]+");
                for (String word : words) {
                    if (!word.isEmpty()) {
                        // Incrementing the count for each word
                        wordCountMap.put(word, wordCountMap.getOrDefault(word, 0) +
1);
                    }
                }
            }

            // Sorting the words by frequency in descending order
            List<Map.Entry<String, Integer>> sortedWords = new
ArrayList<>(wordCountMap.entrySet());
            sortedWords.sort((entry1, entry2) ->
entry2.getValue().compareTo(entry1.getValue()));

            // Displaying the top 5 most frequently occurring words
            System.out.println("Top 5 most frequently occurring words:");
            for (int i = 0; i < Math.min(5, sortedWords.size()); i++) {
                Map.Entry<String, Integer> entry = sortedWords.get(i);
                System.out.println(entry.getKey() + " - " + entry.getValue() + "
occurrences");
            }
        } catch (IOException e) {
```

```
// Handling exceptions related to file operations
System.out.println("Error reading the file: " + e.getMessage());
    }
}
```